

Transformer-Based Generation of Hardware-Constrained QAOA Circuits for MaxCut

Ruairi Hogan

University College Dublin

School of Electrical and Electronic Engineering

Dublin, Ireland

ruairi.hogan@ucdconnect.ie

Abstract—The Quantum Approximate Optimization Algorithm (QAOA), and its adaptive variants, provide a powerful framework for solving combinatorial optimisation problems on near-term quantum devices. It has been recently proposed that generative pre-trained transformers can be used to learn, and reproduce, high quality quantum circuits with less computational cost [1]. In this work, this framework is extended to generate hardware-aware quantum circuits for MaxCut problems, conditioned on both the problem graph, and device connectivity constraints. Training data is generated using ADAPT-QAOA on weighted Erdős–Rényi graphs with seven edges, while enforcing coupling constraints derived from a subgraph of the IBM `ibm_kingston` QPU. The proposed approach enables fast circuit generation without the need for repeated optimisation, demonstrating the potential of language models as learned policies for quantum circuit design under realistic hardware constraints.

I. INTRODUCTION

Quantum computing has become an increasingly relevant field in the decades-long quest to develop a fault-tolerant, low-error machine that can solve certain optimisation problems asymptotically faster than classical computers can do today. Presently, quantum computing architectures are divided into a few large classes, with gate-based approaches being one of the more dominant near-term models [2]. Gate-based quantum computers use a sequence of quantum gates (unitary operations) to manipulate qubits, reflecting how classical computers use logic gates on bits. Various methods have been proposed that are designed to run on a gate-based architecture, which heuristically solve large sets of combinatorial optimisation problems, with the expectation that this architecture will outperform classical computers.

The current quantum machines operate in what is coined as the Noisy Intermediate-Scale Quantum (NISQ) era. These are characterised by devices containing tens to hundreds of qubits, but are not protected by full quantum error correction. While these advancements are very promising, these devices are limited by noise, circuit depth and finite qubit counts [3]. With current quantum computers not yet reliable enough to provide an asymptotic quantum advantage over classical computers, algorithm engineering becomes crucial. Accounting for the constraints imposed by these devices, optimisation-based, or learning-based approaches are widely used. A common method used by these algorithms is a classical-quantum hybrid approach, where a classical optimiser is used alongside a

quantum processor to divide computational tasks and exploit the strengths of both systems.

Variational quantum algorithms (VQAs) have been proposed as a leading strategy for solving combinatorial optimisation problems on NISQ devices. VQAs use parametrised quantum circuits, and rely on a hybrid optimisation loop between classical and quantum components [4]. As a result, the overall performance of VQAs is strongly influenced by circuit structure and parameter initialisation. In the context of QAOA, combinatorial optimisation problems are commonly expressed in the form of a Quadratic Unconstrained Binary Optimisation (QUBO) problem. A canonical example is the MaxCut problem, which is widely used as a benchmark due to its well-defined formulation and easily found classical optima for small instances [5].

The Quantum Approximate Optimisation Algorithm (QAOA) was introduced to heuristically solve combinatorial optimisation problems [6]. It consists of layering quantum circuits to manipulate the qubit states to obtain a target state from the circuit, while a classical optimiser is used to optimise parameters, and obtain a more accurate final state. The expressivity and performance of QAOA can improve with increasing circuit depth; however, performance gains diminish rapidly beyond small depths [7]. Additionally, QAOA is characterised by highly non-convex optimisation landscapes, leading to sensitivity to parameter initialisation and the presence of local minima.

The mixer operator is a critical component of QAOA, as it governs how the algorithm explores the solution space. Alternative mixer constructions have been proposed to improve search efficiency, but often require domain knowledge and can introduce additional overhead on hardware with limited connectivity [8]. To address these limitations, the adaptive derivative assembled problem-tailored QAOA (ADAPT-QAOA) has been proposed as an iterative variant that allows the circuit to adapt to the structure of the specific optimisation instance, aiming to improve solution quality with fewer layers [9].

Machine learning (ML) has become increasingly explored as a means of improving performance in VQAs. Sequence models such as transformers have been proposed as a natural fit for representing quantum circuits, as they allow circuits to be modelled as sequences of tokens, and enable the learning of statistical regularities across problem instances [1]. Leveraging

a pre-trained transformer, namely QAOA-GPT [1], shifts the computational cost from live optimisation to offline training of the model [10]. QAOA-GPT is a GPT-assisted framework for QAOA that uses a generative pre-trained transformer for circuit construction, removing the need for iterative optimisation, and enabling faster generation of problem-specific circuits. Figure 4 shows the significant improvement in circuit generation time for this framework in comparison to simulated ADAPT-QAOA circuit generation.

In this work, a hardware-aware QAOA-GPT framework is developed to generate ADAPT-QAOA circuits for MaxCut problems under realistic device constraints. To the best of our knowledge, this extension has not been previously investigated, and offers a promising approach to reducing error rates in the QAOA-GPT framework. By incorporating hardware connectivity into both dataset generation and model inference, the proposed approach aims to produce physically executable quantum circuits while reducing the computational cost associated with iterative optimisation. In this work, the coupling map of IBM quantum cloud QPUs (Quantum Processing Units) is used to constrain the set of allowable two-qubit interactions. By limiting operations to hardware-native connections, the need for additional qubit routing is reduced, leading to lower circuit depth, and improved robustness to noise. Experimental results show a 2% increase in circuit quality (approximation ratio) when trained with hardware constraints.

II. PROBLEM STATEMENT

Traditional quantum-classical hybrid algorithms, such as QAOA, have a high computational overhead in live circuit generation for each problem instance, due to repeated parameter optimisation and operator selection. This research aims to shift this computational cost to offline learning and reduce circuit generation time significantly with a single forward pass transformer model, while still producing high-quality circuits. To the best of our knowledge, recent approaches to the QAOA-GPT framework have not used device-specific hardware constraints in circuit generation. This work extends the framework to incorporate qubit coupling maps to produce circuits that are both efficient to generate, and more robust to noise during execution.

III. METHODOLOGY

A. Problem Formulation

This work aims to test the effectiveness of these quantum circuits using the MaxCut problem defined on a weighted graph $G = (V, E)$. The objective of MaxCut is to partition the set of vertices into two disjoint subsets such that the total weight of edges across the two subsets is maximised [5]. MaxCut is classified as an NP-hard problem, implying that no known algorithm can solve this on a classical computer in polynomial time. MaxCut is a common benchmark for QAOA and VQA variants. Each vertex of the graph is assigned a binary variable. This problem is mapped to a quantum formulation by converting binary variables to spin variables in the cost Hamiltonian:

$$H_C = \sum_{(i,j) \in E} w_{ij} \frac{1 - Z_i Z_j}{2} \quad (1)$$

where Z_i denotes the Pauli-Z operator acting on qubit i and w_{ij} represents the edge weight. The goal is to prepare a quantum state $|\psi\rangle$ that maximises the expectation value $\langle\psi|H_C|\psi\rangle$, which corresponds to the value of the cut.

B. QAOA

QAOA is a quantum-classical hybrid framework designed to find the approximate solutions to combinatorial optimisation problems on a gate-based quantum computer [11]. Given a MaxCut problem, the cost Hamiltonian encodes an objective function whose ground state corresponds to the optimal solution, and a mixer Hamiltonian H_M allows the exploration of a solution space. The QAOA procedure starts from the uniform superposition state $|+\rangle^{\otimes n}$, and proceeds by sequentially applying p layers of alternating unitary transformations corresponding to the cost and mixing Hamiltonians. These unitaries are defined as

$$U_P(\gamma_i) = e^{-i\gamma_i H_C} \quad \text{and} \quad U_M(\beta_i) = e^{-i\beta_i H_B}, \quad (2)$$

respectively.

After the application of p such layers, the resulting QAOA state is given by

$$|\gamma, \beta\rangle = U_M(\beta_p)U_P(\gamma_p) \cdots U_M(\beta_1)U_P(\gamma_1)|+\rangle^{\otimes n}, \quad (3)$$

where $\gamma = \{\gamma_1, \dots, \gamma_p\}$ and $\beta = \{\beta_1, \dots, \beta_p\}$ denote the corresponding sets of variational parameters.

The cost Hamiltonian expectation value is evaluated by the formula

$$\langle H_C \rangle = \langle \gamma, \beta | H_C | \gamma, \beta \rangle. \quad (4)$$

For a given depth, the QAOA state can be written as

$$|\psi(\gamma, \beta)\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n} \quad (5)$$

C. ADAPT-QAOA

The ADAPT-QAOA (Adaptive Derivative Assembled Problem-Tailored Quantum Approximate Optimisation Algorithm) is a variant of QAOA that constructs the quantum circuit adaptively rather than using a fixed ansatz. Instead of applying a predetermined sequence of cost and mixer operators, the algorithm selects operators iteratively based on their contribution to improving the objective function.

The algorithm begins with an initial state and a predefined pool of candidate operators. At each iteration, the gradient of the cost function with respect to each operator in the pool is evaluated. The operator with the largest gradient is chosen, and added to the circuit.

This selection can be expressed as

$$A^* = \arg \max_{A_k \in \mathcal{P}} |g_k| \quad (6)$$

where $\mathcal{P}$ is the operator pool and g_k represents the gradient associated with operator A_k .

After selecting an operator, the variational parameters of the circuit are optimised using a classical optimiser. This process

is repeated, resulting in a circuit that is constructed layer-by-layer, where each additional parameter is chosen to maximise the improvement in the objective function.

The resulting ADAPT-QAOA state can be written as

$$|\psi(\theta)\rangle = e^{-i\theta_L A_L} \dots e^{-i\theta_2 A_2} e^{-i\theta_1 A_1} |+\rangle^{\otimes n} \quad (7)$$

where $\theta = (\theta_1, \dots, \theta_L)$ are the optimised parameters and A_i are the selected operators.

In this work, ADAPT-QAOA is used as a data generation procedure, producing high-quality circuits that are later used to train the GPT model.

D. Dataset Generation

Training data is generated by using simulated ADAPT-QAOA circuits to solve MaxCut instances of randomly generated graphs.

The quantum algorithm is simulated using the IBM Qiskit Python Framework [12]. At each iteration of ADAPT-QAOA, candidate operators are evaluated, the best operator is selected, and the circuit parameters are optimised. This results in a sequence of operators and parameters defining the constructed circuit.

$$\alpha = \frac{\langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle}{\text{OPT}(G)}, \quad (8)$$

where H_C denotes the cost Hamiltonian derived from the Max-Cut instance, and $\text{OPT}(G)$ represents the best-known solution value obtained using classical heuristic methods. The number of qubits considered in the simulation is limited by computational cost and thus is relatively small. As a result, the classical heuristic methods achieve near-optimal solutions for the MaxCut problem.

The data is represented as a structured token sequence so that the language model can condition circuit generation on both the target hardware and the MaxCut instance. Each example begins with a beginning-of-sequence token, followed by the hardware description, the MaxCut graph, and the circuit. In general, the format is

```
<bos> <name_of_QPU> <hardware_graph> ...
<end_of_hardware_graph> <maxcut_graph>
... <end_of_maxcut_graph> <circuit> ...
<end_of_circuit>.
```

For example, the MaxCut problem is encoded as

```
<maxcut_graph> (0, 3) 0.96 (0, 4) 0.18
(1, 3) 0.93 ...<end_of_maxcut_graph>,
while the generated circuit is written as a sequence of
layer tokens, such as <circuit> <new_layer_1>
55 0.0 0.79 <new_layer_2> 44 0.0 -0.79
...<end_of_circuit>.
```

E. Hardware-aware Constraints

To ensure that the circuits generated by the model are physically realisable, this research has extended the QAOA-GPT framework [1] by leveraging hardware constraints in the data generation. A qubit coupling map corresponding to an IBM quantum device [13] is used to restrict the set of allowable two-qubit interactions. Operators acting on pairs of

qubits that are not connected in the coupling map are excluded from the candidate operator pool. This aims to ensure that the generated circuits are compatible with the connectivity constraints of the QPU, eliminating the need for additional routing and aiming to reduce errors. A coupling map is taken from IBM's Kingston QPU, with a 7-qubit subgraph used for these circuits.

F. Model Architecture and Training

This research utilises a modified version of nanoGPT [14], following the QAOA-GPT model of Tyagin et al. [1]. This is a decoder-only transformer architecture. It consists of a stack of transformer blocks, each containing masked multi-head self-attention and feed-forward layers, enabling autoregressive sequence generation. The model is trained using a next-token prediction objective, where each token is generated conditioned on all previous tokens in the sequence. This allows the model to generate circuits as a form of sequence modelling.

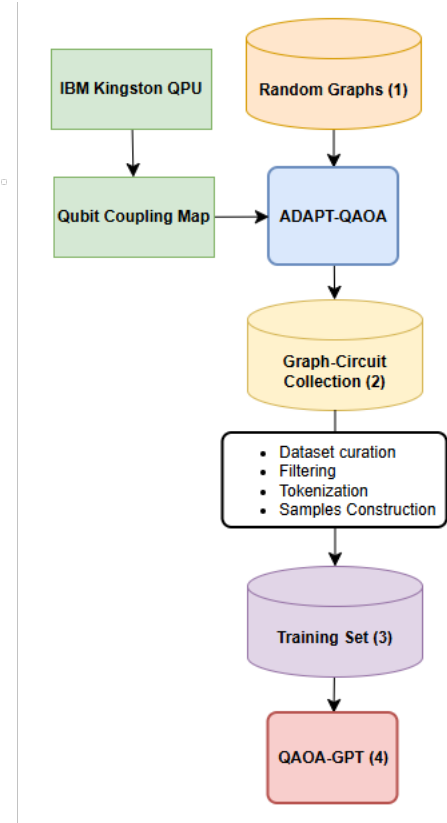


Fig. 1: Schematic representation of QAOA-GPT framework with hardware-aware extension. Random graphs are processed by ADAPT-QAOA under hardware constraints of IBM's Kingston QPU qubit coupling map. These form a graph-circuit collection which is tokenised and transformed to build the training set for the GPT Model.

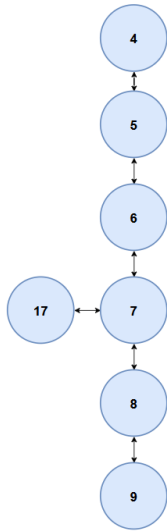


Fig. 2: Qubit Coupling Sub-Map from IBM Kingston QPU of qubits used for testing QAOA-GPT [13].

IV. EXPERIMENTAL SETUP

A. Problem Instances

MaxCut instances were generated using the Erdős-Rényi random graph model $G(n, p)$ with $n = 7$ nodes. For each instance, the edge probability p was sampled uniformly from the interval $[0.3, 0.9]$. This range was chosen to avoid trivial sparse graphs while maintaining sufficient structural diversity across the dataset. This prevents the graphs from being too sparse or too dense, thereby preventing the problem from becoming too easy, as many cuts produce the same value.

Only connected graphs were retained. Each edge (i, j) was assigned a weight w_{ij} sampled uniformly from the interval $(0, 1]$, resulting in weighted MaxCut instances.

B. Hardware Constraints

All circuits were generated under connectivity constraints derived from a 7-qubit subgraph of the IBM `ibm_kingston` device. The logical qubits were mapped to physical qubits $\{4, 5, 6, 7, 8, 9, 17\}$, resulting in the coupling map

$$(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (3, 6). \quad (9)$$

Two-qubit operators were restricted to act only on qubit pairs present in this coupling map, ensuring that all generated circuits are compatible with the target hardware.

[13]

C. Dataset Generation

Training data was generated using the ADAPT-QAOA algorithm with a maximum circuit depth of 9. For each graph, multiple initial values of $\gamma_0 \in \{0.01, 0.1, 0.5, 1.0\}$ were used to improve solution quality.

The optimal MaxCut value was computed via brute force enumeration over all 2^7 bitstrings. Circuits were evaluated using the approximation ratio

$$\alpha = \frac{\langle \psi | H_C | \psi \rangle}{\text{OPT}(G)}. \quad (10)$$

Only circuits achieving $\alpha \geq 0.97$ were retained. A total of 12'000 valid circuits were collected.

D. Operator Pool

The operator pool was defined using the PDUAL setting, consisting of single-qubit Pauli operators $\{X_i, Y_i, Z_i\}$ for all qubits, and two-qubit Pauli strings $\{B_i C_j\}$ where (i, j) is an allowed hardware edge and $B, C \in \{X, Y, Z\}$.

This ensures that all candidate operators are compatible with the hardware connectivity constraints.

E. Model Configuration

A decoder-only transformer model based on NanoGPT was used. The model consists of multiple transformer layers with masked multi-head self-attention and feed-forward blocks. The model was trained to predict the next token in the sequence given the previous context.

F. Training Procedure

The model was trained on the generated dataset of 12'000 graph-circuit sequences using a standard next-token prediction objective with cross-entropy loss. During inference, operator masking was applied to restrict the model's output to hardware-valid operators, ensuring that generated circuits remain physically executable.

G. Control Setup

In addition to the hardware-aware model, a baseline GPT model was trained without incorporating hardware coupling constraints. This model serves as a control, allowing circuits to be generated without restricting two-qubit interactions to the device connectivity graph. The generated circuits can then be transpiled and executed on the IBM quantum cloud platform, enabling a direct comparison between hardware-constrained and unconstrained approaches. This setup allows the impact of incorporating coupling maps into the generation process to be evaluated. The control GPT was setup using the same process and trained on 12'000 MaxCut graphs and circuits.

V. COMPUTATIONAL RESULTS

A. Evaluation Metrics

This section evaluates the performance of the proposed approach using the metrics defined in Section III-B. In particular, the approximation ratio is used to assess the quality of generated circuits, while runtime is used to compare the computational efficiency of circuit generation methods. These metrics are applied consistently across all experiments to enable a direct comparison between ADAPT-QAOA, the control GPT model, and the hardware-aware GPT model.

B. Accuracy of Hardware-Aware GPT Circuits

To report the performance of the model, the AR of the generated circuits are evaluated. First, 100 random graphs are generated using Erdős-Rényi random graph model as described in Section IV-A. For each of these graphs, the hardware-aware transformer model generated 10 circuits. These circuits are simulated with ideal conditions using IBM’s Qiskit Python framework [12]. The best AR and the mean AR of these circuits are then evaluated as shown in Figure 3. The highest AR averages 0.9242 ± 0.0785 , and the mean AR averages 0.8588 ± 0.0767 . This shows that while there exists a gap between average and highest AR from each sample, the model is capable of producing strong candidate solutions without the need for iterative optimisation for this relatively low number of qubits.

C. Runtime Comparison

This experiment evaluates the proposal that the live computational cost of circuit generation will be reduced with the QAOA-GPT method compared to ADAPT-QAOA. The metric of evaluation is the runtime to generate 20 circuits with an approximation ratio of at least 80%. Although the ADAPT-QAOA circuits are generated via simulation on a CPU, the generation time will be less than performing ADAPT-QAOA on a QPU as individual circuit executions on quantum hardware occur on microsecond timescales; the need for repeated measurements and system-level overheads results in significantly higher effective runtimes from a user perspective. Both the circuit simulation and GPT circuit creation are performed on a 13th Gen Intel(R) Core(TM) i5-1335U CPU. The results of the generation time are shown in Figure 4. The ADAPT-QAOA approach requires a mean runtime of 9.10 seconds per instance, whereas the GPT model generates circuits in an average of 0.52 seconds. This corresponds to a mean speed-up of approximately 17.4 $\times$. These results highlight the significant reduction in computational cost of live circuit generation as circuits are generated in a single forward pass of the model instead of an iterative process. While these circuits are relatively small to simulate, as the size of the graphs increase, the time required to generate circuits will increase rapidly. The caveat to this approach in runtime comparison is that when the size of the graphs increases, the training data will eventually become computationally infeasible to produce at scale, thus the model will need to resort to being trained on ADAPT-QAOA circuits evaluated on a physical QPU. While the accuracy of this method will reduce as a result, the runtime for circuit generation should still be significantly faster using the QAOA-GPT approach; the training data generation will be significantly slower.

D. Impact of Hardware Constraints on Performance

This experiment evaluates the impact of incorporating hardware connectivity constraints into the GPT-based circuit generation process. The performance of a hardware-aware GPT model is compared against a baseline control model that does not enforce coupling map constraints. Each model is used to

generate 20 circuits from a fixed set of 20 MaxCut instances. When simulating these circuits in ideal conditions, the control model achieves a mean approximation ratio α of 0.8990 with a standard deviation of 0.0505. The hardware-constrained model achieves a mean approximation ratio of 0.8923 and a standard deviation of 0.0923. The close agreement between these values provides a strong basis for comparison, as it suggests that both models exhibit similar performance in the absence of hardware noise. All of these 40 circuits are then run using IBM’s Kingston QPU on the IBM cloud quantum framework [13]. The results of mean AR across these circuits are shown in Figure 5. The hardware-aware model achieves a higher mean approximation ratio of 0.696 compared to 0.674 for the control model, indicating that the inclusion of hardware constraints does not degrade performance and may in fact improve solution quality. While the performance improvement of 0.22 is modest, with only 7 qubits being simulated and only 1 non-linear qubit in the coupling map, the advantage is still evident.

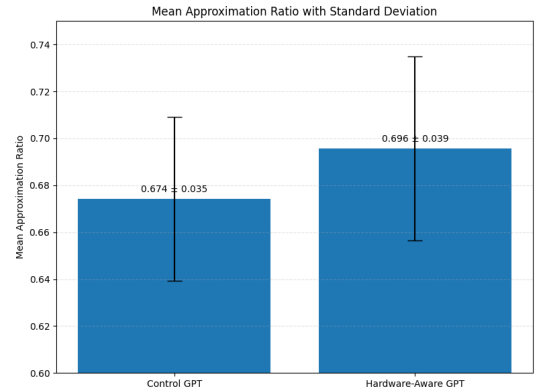


Fig. 5: Performance of ADAPT-QAOA circuits generated with hardware-constrained model vs. unconstrained model, evaluated on IBM_Kingston QPU. The mean AR α of the hardware-constrained model is 0.022 higher than the mean AR of the unconstrained model.

VI. CONCLUSION

This work presented a hardware-aware QAOA-GPT framework that incorporates a real QPU coupling map to reduce routing and SWAP counts for circuit generation. This allows circuits to be generated for MaxCut problem instances under realistic hardware constraints. Experimental results demonstrate that the model achieves high approximation ratios while providing a significant reduction in runtime compared to ADAPT-QAOA, with a speed-up of over 17 $\times$. Furthermore, the inclusion of hardware constraints was shown to improve practical circuit quality without sacrificing performance. These results highlight the potential of language models as efficient, hardware-aware circuit generation tools for near-term quantum computing, however, there exists limitations in efficiently creating training data for very complex optimisation problems, as a classical computer can only simulate approximately 55 qubits presently.

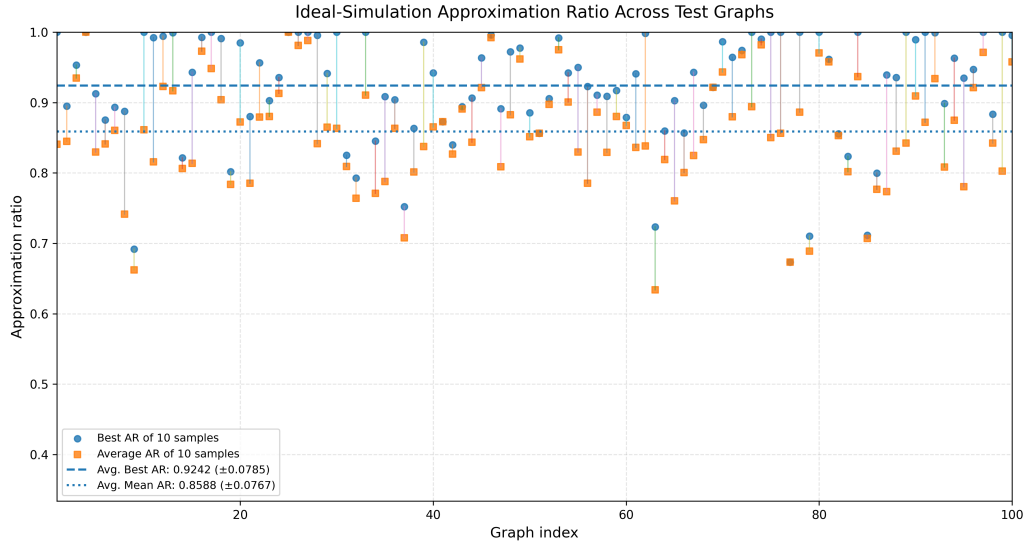


Fig. 3: Performance Distribution of Hardware-aware QAOA-GPT on 100 7-qubit MaxCut instances with a max depth $p_{max} = 15$. For each graph, the model generated 10 circuits. The highest AR of the 10 samples (blue) are shown, averaging 0.9242 ± 0.0785 , and the mean AR of the 10 samples (orange) are shown, averaging 0.8588 ± 0.0767 .

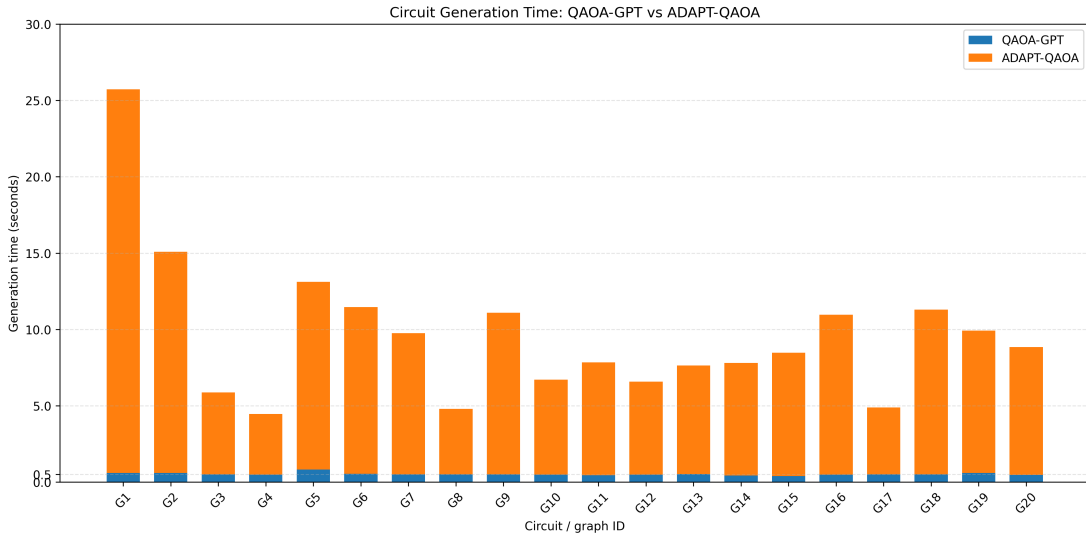


Fig. 4: Runtime for circuit generation of 7-qubit problem instances from Hardware-aware QAOA-GPT compared to simulated ADAPT-QAOA. The QAOA-GPT runtime for each circuit ID (blue) and the ADAPT-QAOA runtime (orange) are shown. The ADAPT-QAOA approach requires a mean runtime of 9.10 seconds per circuit, whereas the GPT model generates circuits in an average of 0.52 seconds.

REFERENCES

- [1] I. Tyagin, M. H. Farag, K. Sherbert, K. Shirali, Y. Alexeev, and I. Safro, "QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits," in *2025 International Conference on Quantum Computing and Engineering*, 4 2025.
- [2] T. Fankhauser, M. E. Solèr, R. M. Füchslin, and K. Stockinger, "Multiple query optimization using a gate-based quantum computer," *IEEE Access*, vol. 11, 2023.
- [3] M. AbuGhanem and H. Eleuch, "Nisq computers: A path to quantum supremacy," *IEEE Access*, vol. 12, 2024.
- [4] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [5] F. Rendl, G. Rinaldi, and A. Wiegale, "Solving max-cut to optimality by intersecting semidefinite and polyhedral relaxations," *Mathematical Programming*, vol. 121, no. 2, pp. 307–335, 2010.
- [6] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, 2019. [Online]. Available: <https://www.mdpi.com/1999-4893/12/2/34>
- [7] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance,

- mechanism, and implementation on near-term devices,” *Phys. Rev. X*, vol. 10, p. 021067, Jun 2020. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevX.10.021067>
- [8] F. G. Fuchs and R. P. Bassa, “LX-mixers for QAOA: Optimal mixers restricted to subspaces and the stabilizer formalism,” *Quantum*, vol. 8, p. 1535, Nov. 2024. [Online]. Available: <https://doi.org/10.22331/q-2024-11-25-1535>
 - [9] N. Yanakiev, N. Mertig, C. K. Long, and D. R. M. Arvidsson-Shukur, “Dynamic adaptive quantum approximate optimization algorithm for shallow, noise-resilient circuits,” *Phys. Rev. A*, vol. 109, p. 032420, Mar 2024. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevA.109.032420>
 - [10] L. Sunny, A. Rijal, and G. Siopsis, “Extending qaoa-gpt to higher-order quantum optimization problems,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.07391>
 - [11] L. Zhu, H. L. Tang, G. S. Barron, F. A. Calderon-Vargas, N. J. Mayhall, E. Barnes, and S. E. Economou, “Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer,” *Phys. Rev. Res.*, vol. 4, p. 033029, Jul 2022. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevResearch.4.033029>
 - [12] IBM Quantum, “Qiskit: An Open-Source Framework for Quantum Computing,” <https://www.ibm.com/quantum/qiskit>.
 - [13] —, “IBM Quantum Cloud,” <https://quantum.cloud.ibm.com/>.
 - [14] A. Karpathy, “nanoGPT,” <https://github.com/karpathy/nanoGPT>, 2023, gitHub repository.